

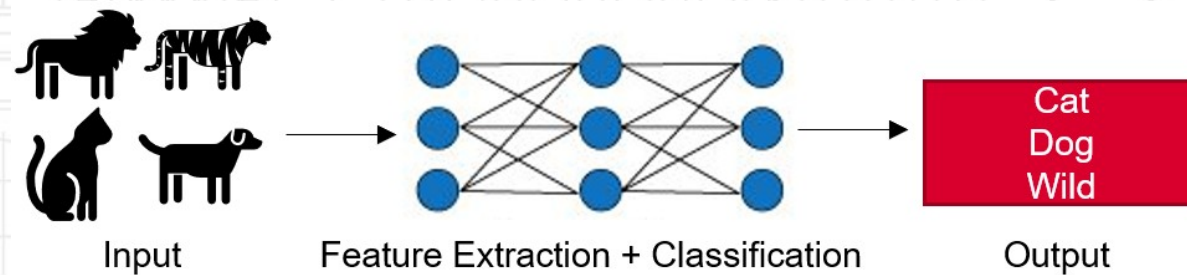


Module 2

Multi-Class Classification



Multi-Class Classification



Types of Classification Problems

- The 3 main classification problems are:

Binary Classification



- Spam
- Not spam

Multiclass Classification



- Dog
- Cat
- Horse
- Fish
- Bird
- ...

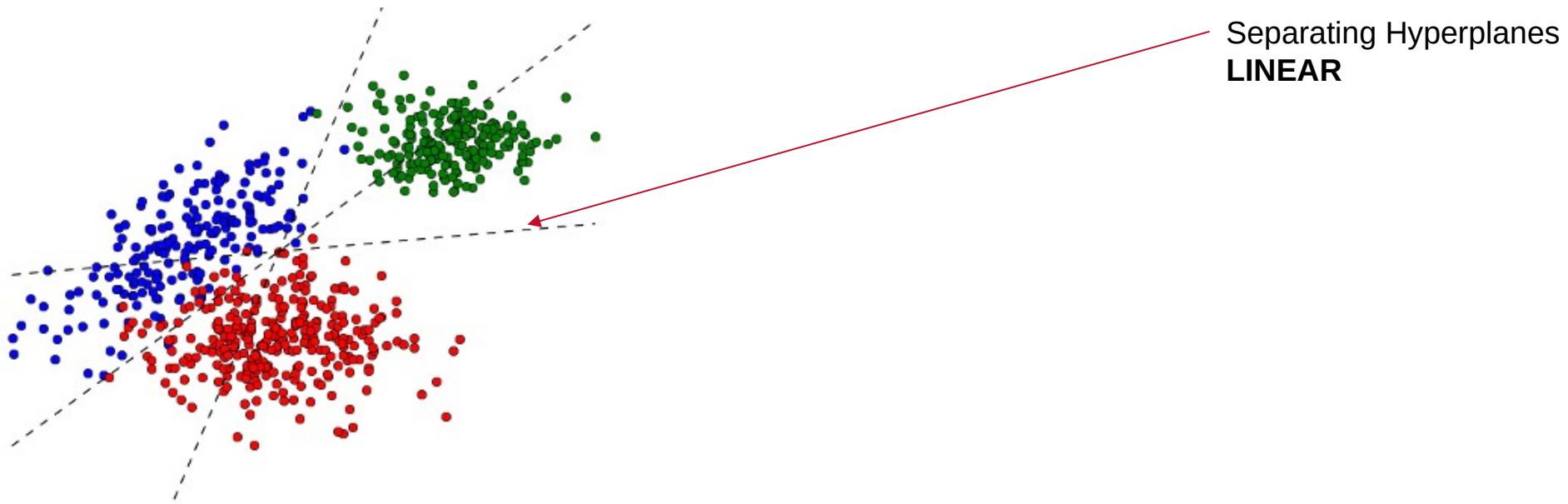
Multi-label Classification



- Dog
- Cat
- Horse
- Fish
- Bird
- ...

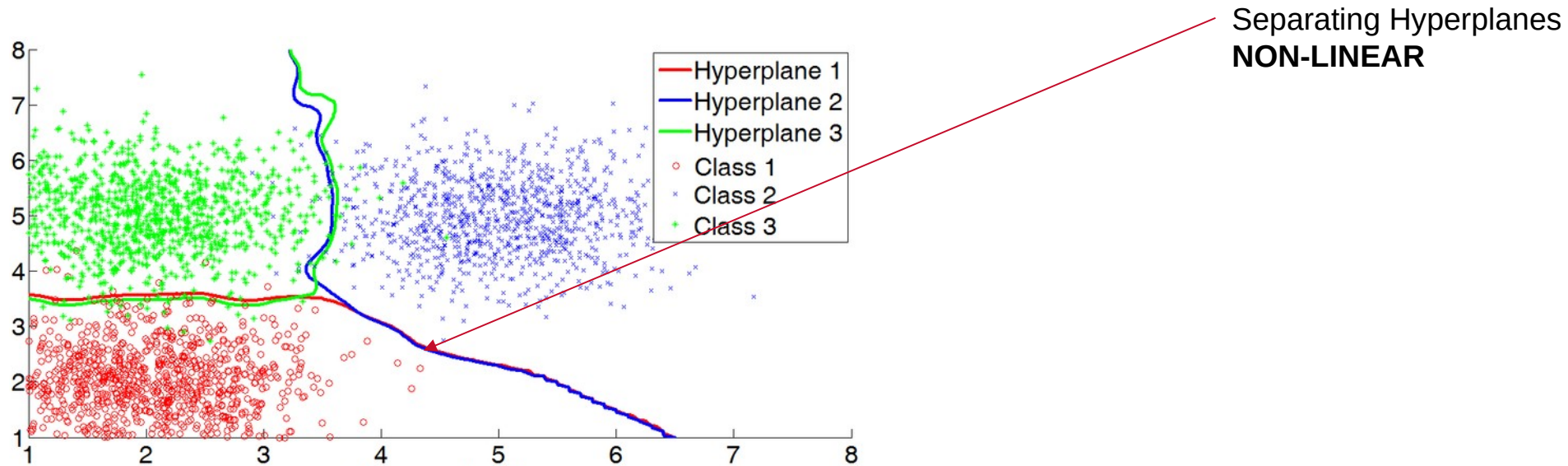
Multi-Class Classification

□ is a classification task with more than two classes.



Multi-Class Classification

□ is a classification task with more than two classes.



Multi-Class Classification

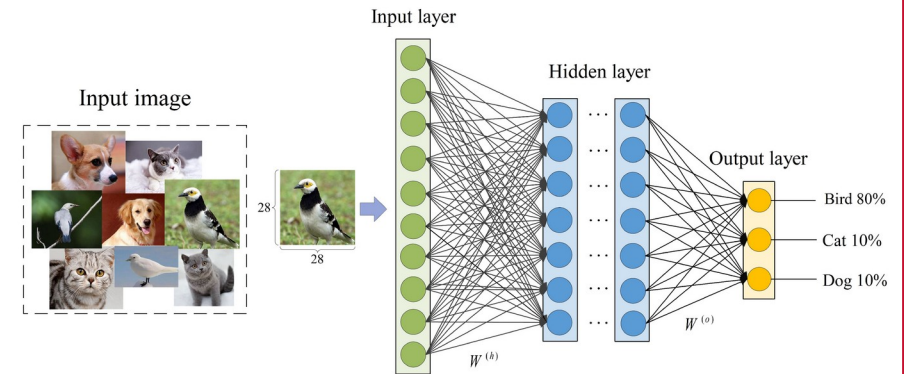
Feature Matrix (X)

n_features $\rightarrow$

$\leftarrow$ n_samples					

Target Vector (y)

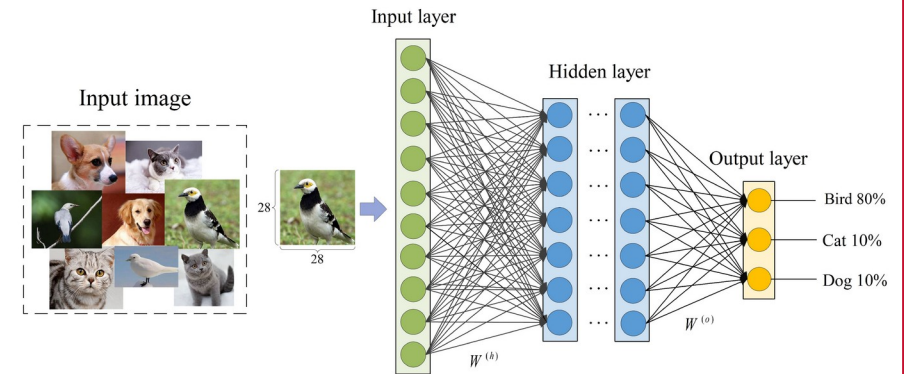
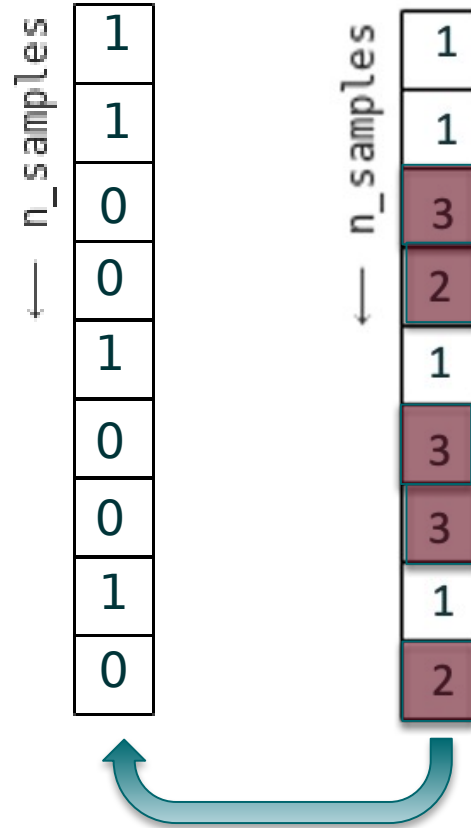
$\leftarrow$ n_samples	1
	1
	3
	2
	1
	3
	3
	1
	2



Multi-Class Classification

Feature Matrix (X)
n_features $\rightarrow$

$\leftarrow$ n_samples										



Multi-Class Classification

Feature Matrix (X)
n_features $\rightarrow$

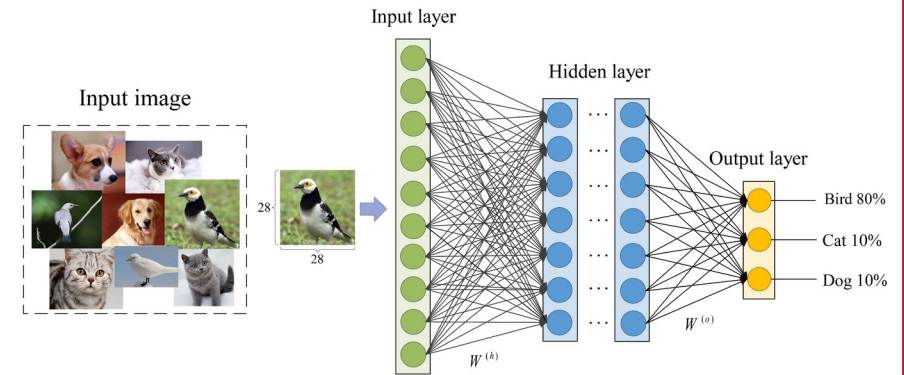
$\leftarrow$ n_samples

$\leftarrow$ n_samples

0
0
0
1
0
0
0
0
0
1

$\leftarrow$ n_samples

1
1
3
2
1
3
3
1
2



Multi-Class Classification

Feature Matrix (X)
n_features $\rightarrow$

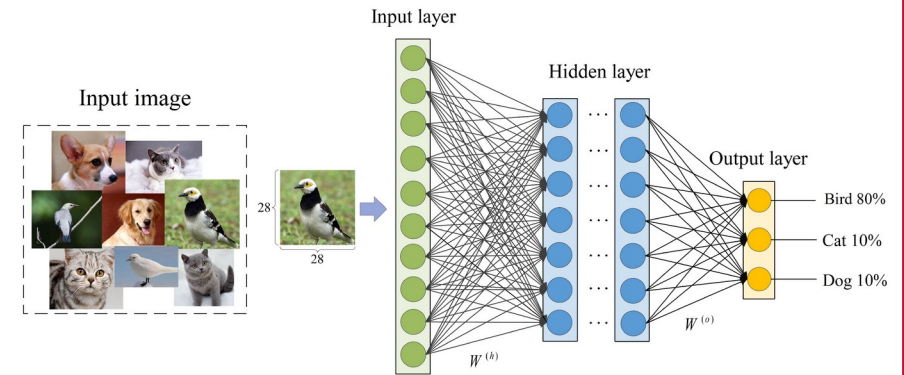
$\leftarrow$ n_samples

$\leftarrow$ n_samples

0
0
1
0
0
1
1
0
0

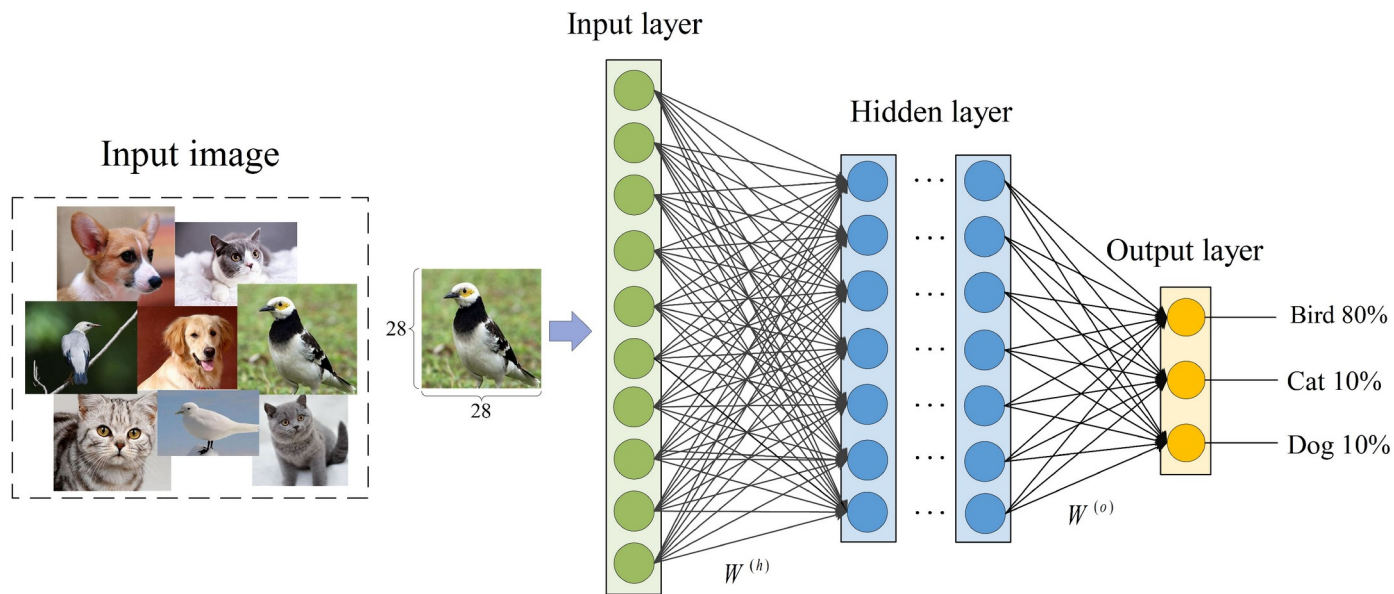
$\leftarrow$ n_samples

1
1
3
2
1
3
3
1
2



Multi-Class Classification - Propensities

- Linear and non-linear predictors output the probability of belonging to a particular class ($\hat{p}$)



Categorical Cross-Entropy

- The most common **loss** function in multi-class classification

$$-\sum_k y_k \log \hat{p}_k$$

How do we determine the **distance** between this two matrices?

	DOG	CAT	BIRD
	0	1	0
	1	0	0
	0	1	0
	0	0	1

p_dog	p_cat	p_bird
0.05	0.90	0.05
0.85	0.10	0.05
0.80	0.10	0.10
0.10	0.15	0.75

Categorical Cross-Entropy

- The most common **loss** function in multi-class classification

$$-\sum_k y_k \log \hat{p}_k$$

DOG	CAT	BIRD
0	1	0
1	0	0
0	1	0
0	0	1

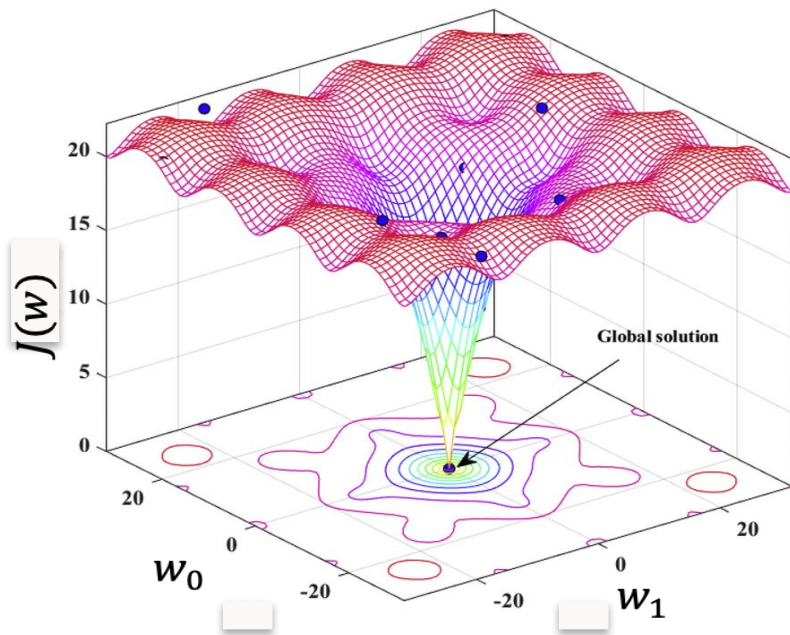
p_dog	p_cat	p_bird
0.05	0.90	0.05
0.85	0.10	0.05
0.80	0.10	0.10
0.10	0.15	0.75

$$-(0 \log 0.05 + 1 \log 0.90 + 0 \log 0.05) = 0.1054$$

Categorical Cross-Entropy

- The cost function is averaged over the observations

$$J(w) = -\frac{1}{n} \sum_i \sum_k y_k \log \hat{p}_k$$



Combining Predictions

- We use the softmax function to combine predictions

Feature Matrix (X)
n_features →

← n_samples

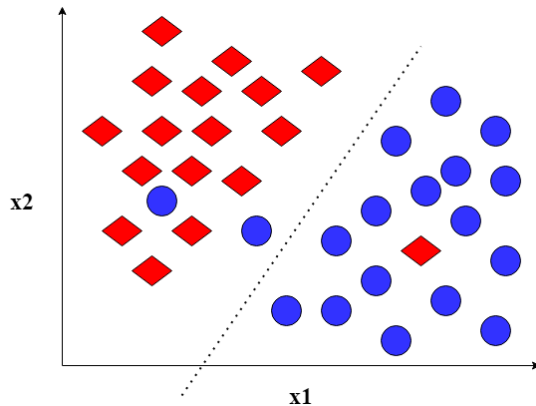
$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix} \rightarrow \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \rightarrow \begin{bmatrix} 0.02 \\ 0.90 \\ 0.05 \\ 0.01 \\ 0.02 \end{bmatrix}$$

Binary Classification Algorithms?

- ❑ We can still use binary classification algorithms in multi-class classification
- ❑ Strategies: One-Vs-All (One-Vs-Rest) and One-Vs-One

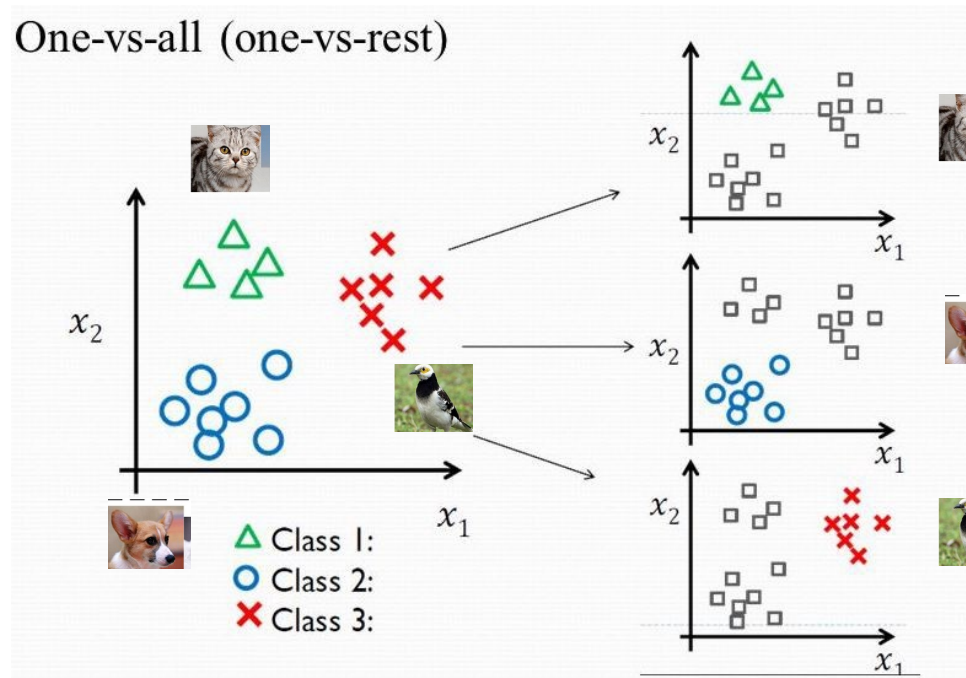
Example: **LOGISTIC REGRESSION**

$$\hat{p} = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \dots)}} = \frac{1}{1 + e^{Xw}}$$



One-Vs-All (One-Vs-Rest)

- ❑ This approach implies fitting the model multiple times
- ❑ In each fit the binary classifier compares one class against the rest



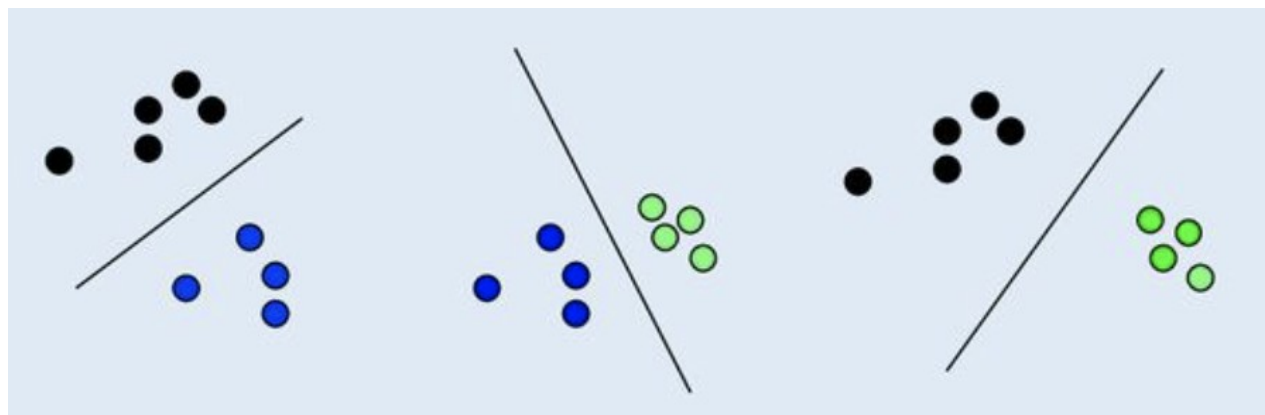
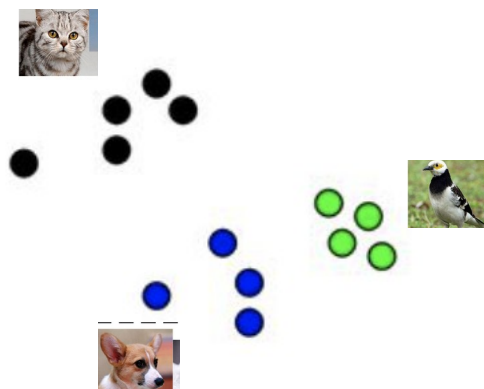
$$\hat{p} = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \dots)}} = \frac{1}{1 + e^{Xw}}$$

$$\hat{p} = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \dots)}} = \frac{1}{1 + e^{Xw}}$$

$$\hat{p} = \frac{1}{1 + e^{-(w_0 + w_1 x_1 + \dots)}} = \frac{1}{1 + e^{Xw}}$$

One-Vs-One

- ❑ This approach implies fitting the model multiple times
- ❑ In each fit the binary classifier compares one class against each other class



$$\hat{p} = \frac{1}{1 + e^{xw}}$$



$$\hat{p} = \frac{1}{1 + e^{xw}}$$



$$\hat{p} = \frac{1}{1 + e^{xw}}$$

IRIS Dataset

- ❑ Classical ML dataset

iris setosa



petal

sepal

iris versicolor



petal

sepal

iris virginica

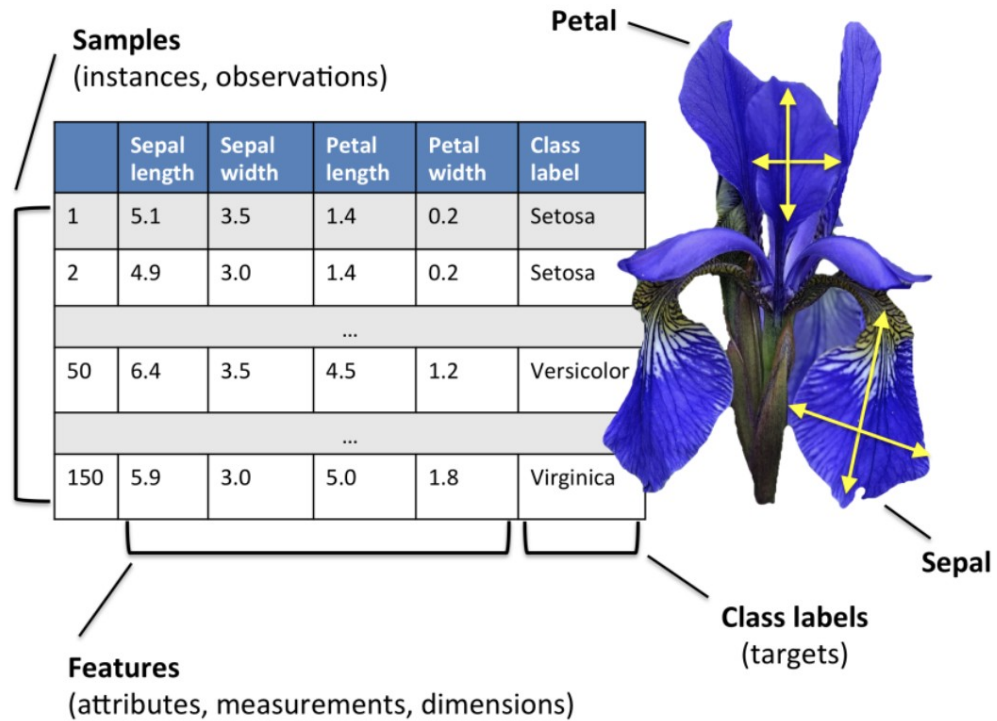


petal

sepal

IRIS Dataset

□ Classical ML dataset





Python

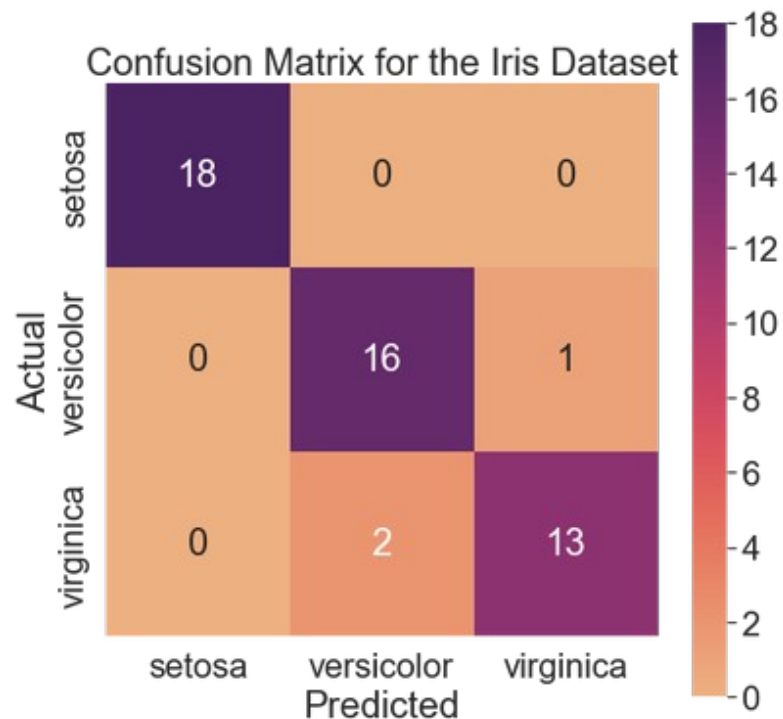
Metrics

- ❑ Used to determine if model is performing well

Scoring	Function	Comment
Classification		
'accuracy'	<code>metrics.accuracy_score</code>	
'balanced_accuracy'	<code>metrics.balanced_accuracy_score</code>	
'top_k_accuracy'	<code>metrics.top_k_accuracy_score</code>	
'average_precision'	<code>metrics.average_precision_score</code>	
'neg_brier_score'	<code>metrics.brier_score_loss</code>	
'f1'	<code>metrics.f1_score</code>	for binary targets
'f1_micro'	<code>metrics.f1_score</code>	micro-averaged
'f1_macro'	<code>metrics.f1_score</code>	macro-averaged
'f1_weighted'	<code>metrics.f1_score</code>	weighted average
'f1_samples'	<code>metrics.f1_score</code>	by multilabel sample
'neg_log_loss'	<code>metrics.log_loss</code>	requires <code>predict_proba</code> support
'precision' etc.	<code>metrics.precision_score</code>	suffixes apply as with 'f1'
'recall' etc.	<code>metrics.recall_score</code>	suffixes apply as with 'f1'
'jaccard' etc.	<code>metrics.jaccard_score</code>	suffixes apply as with 'f1'
'roc_auc'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovr'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovo'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovr_weighted'	<code>metrics.roc_auc_score</code>	
'roc_auc_ovo_weighted'	<code>metrics.roc_auc_score</code>	

Metrics Review: Confusion Matrix

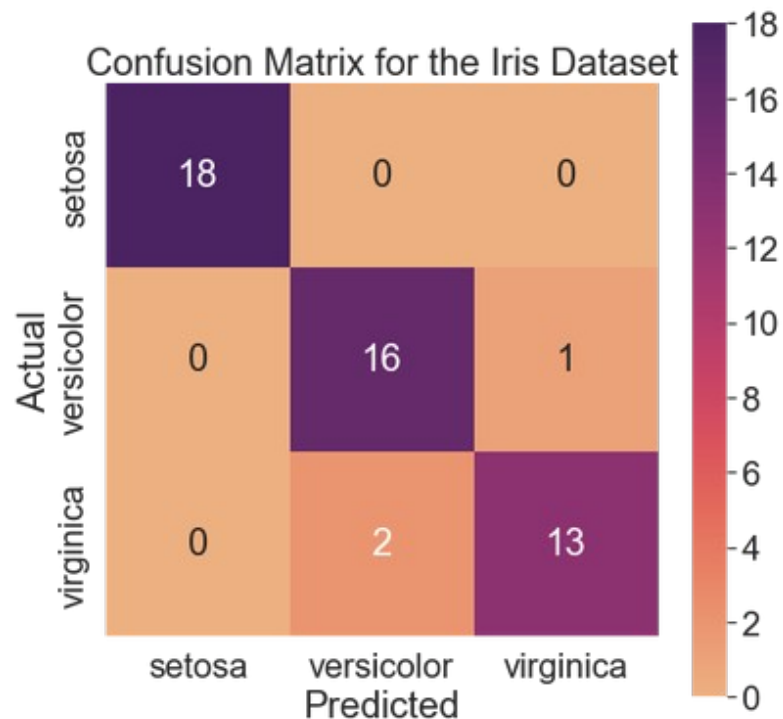
- ❑ The confusion matrix gives a summary of overall performance
- ❑ Accuracy can be derived as the fraction of correct predictions



$$\text{accuracy} = \frac{18 + 16 + 13}{50}$$

Metrics Review: Recall (Sensitivity)

- ❑ Detection rate for all classes
- ❑ We can use macro average (average by the number of classes).



$$\text{recall setosa} = \frac{18}{18} = 1$$

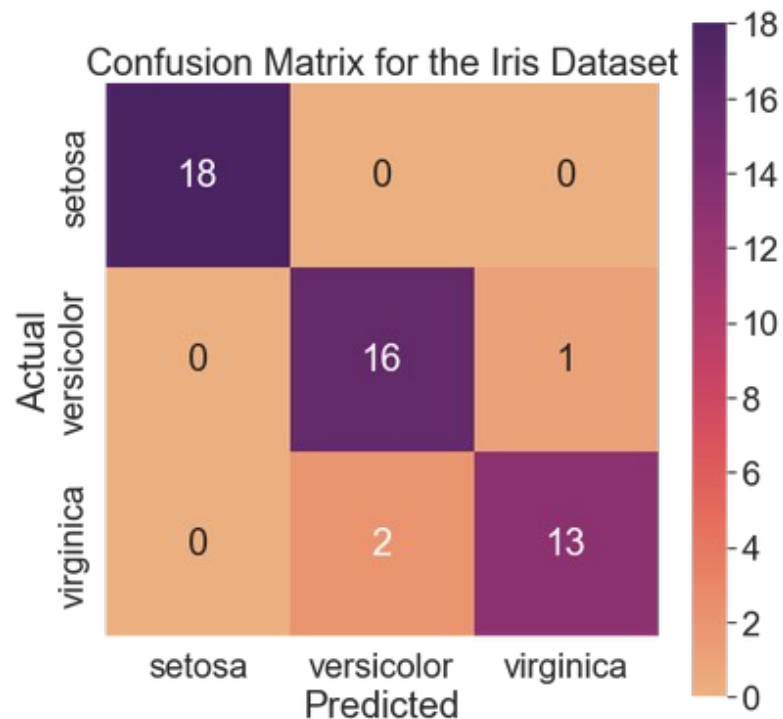
$$\text{recall versicolor} = \frac{16}{17} = 0.9412$$

$$\text{recall virginica} = \frac{13}{15} = 0.8667$$

$$\text{recall macro} = \frac{1 + 0.9412 + 0.8667}{3} = 0.9360$$

Metrics Review: Recall (Sensitivity)

- ❑ Detection rate for all classes
- ❑ We can use micro average (sum true detections over all obs).



$$\text{recall micro} = \frac{18 + 16 + 13}{18 + 17 + 15} = 0.9400$$



Python

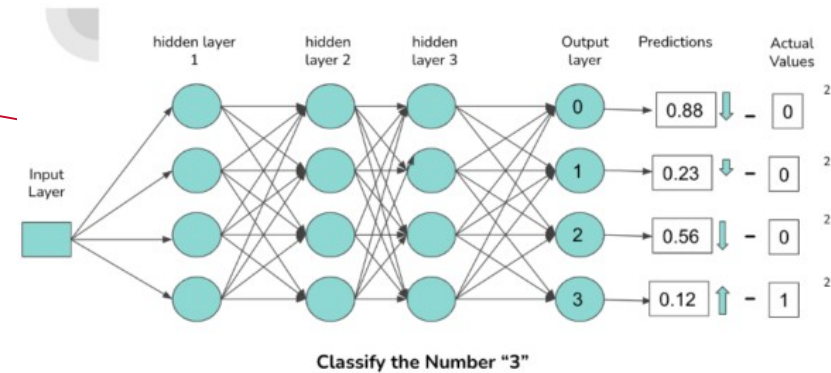
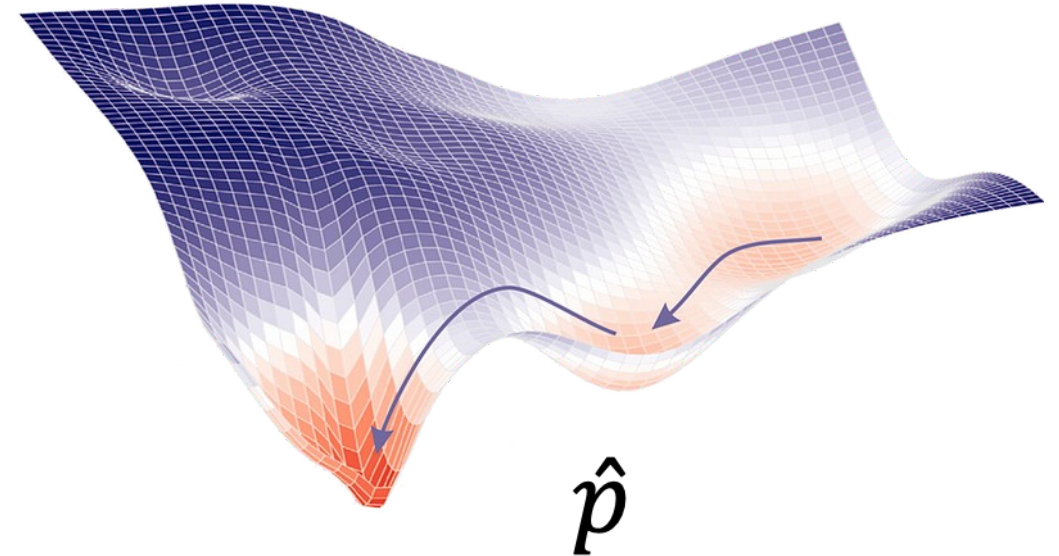
Common Cost Functions Neural Nets

- Categorical Cross-Entropy
- Loss Function

$$J(w) = - \sum_k y_k \log \hat{p}_k$$

- Gradient

$$\nabla J(w) = \hat{p} - y$$



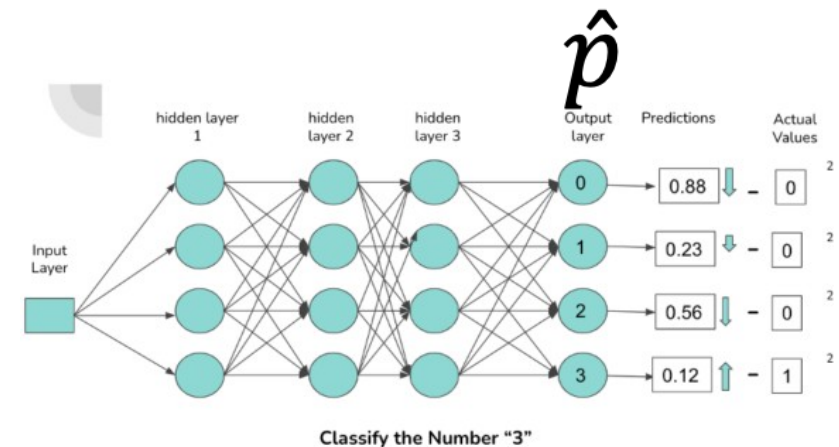
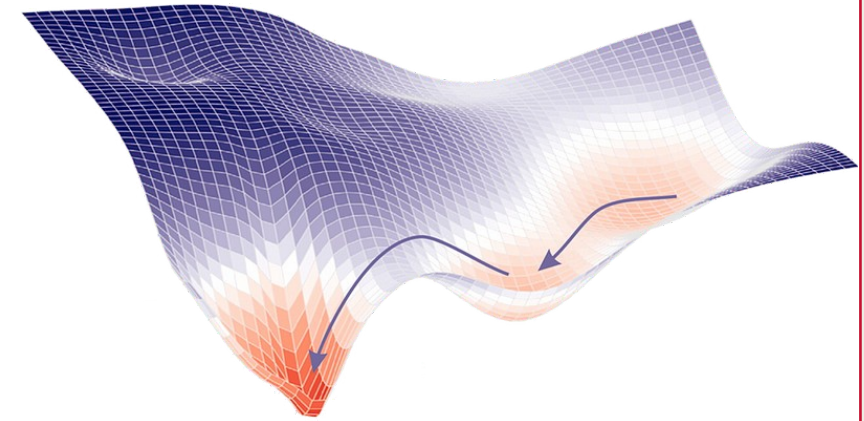
Activation Functions Neural Nets

- Common Activations on Layers

- ReLu
- Tanh
- Sigmoid

- Output Layer Activation

- Softmax





Python